

1. Benchmark Videosunun Hazırlanması

Hazır bir video bulmaktan ziyade, sistemin kullanılacağı gerçek ortama benzer bir video çekmenizi şiddetle tavsiye ederim. Tezinizin savunmasında, "kendi senaryomuzu kurguladık" demek çok daha profesyonel durur.

Video Çekim Senaryosu (Öneri):

Telefonunuzu veya kamerayı, Raspberry Pi'yi konumlandıracağınız açığa (örneğin bir kapı girişi yüksekliğine) sabitleyin ve şu sekansları içeren **1 - 1.5 dakikalık** tek parça bir MP4 videosu çekin:

- (0-15 sn):** Sisteme kayıtlı 1. kişi kameraya doğru yürür, farklı açılardan (sağ, sol, yukarı) bakar ve çıkar.
- (15-30 sn):** Sisteme kayıtlı 2. kişi aynı işlemi yapar ama bu kez gözlük/şapka gibi bir aksesuar takar.
- (30-45 sn):** Sisteme kayıtlı **olmayan** (Unknown) bir yabancı kameraya bakar. (Sistemin bu kişiyi reddettiğini kanıtlamak için).
- (45-60 sn):** Kayıtlı olan ve olmayan kişi **aynı anda** kadrja girer (Multi-face detection ve performans düşüşünü ölçmek için).
- (60-75 sn):** Ortamın ışığı anlık olarak değiştirilir (varsa bir lamba açıp kapatılır) ve bir kişi kameradan hızlıca geçer (Motion blur ve aydınlatma testi).

2. Kodun Sınıflara (Classes) Ayrılması ve Modüler Yapı

Mevcut monolitik (tek parça) yapıyı aşağıdaki gibi dosyalara bölmek, her optimizasyon adımını izole bir şekilde test etmenizi sağlayacaktır.

Önerilen Dosya / Klasör Yapısı:

/thesis_project

```
|
|— config.py          # Tüm eşik değerleri (Thresholds), path'ler ve ayarlar.
|— face_detector.py   # YOLO modelini yükleyen ve Bounding Box dönen sınıf.
|— face_recognizer.py # InsightFace'i yöneten, Embedding çıkaran sınıf.
|— benchmark.py       # Videoyu okuyan ve metrikleri hesaplayan değerlendirme betiği.
|— main.py            # RPi5 kamerasından canlı çalışacak asıl prodüksiyon kodu.
```

Örnek Taslak (Skeleton) Kodlar:

face_detector.py (Yalnızca Detection İşlemleri)

```
from ultralytics import YOLO
```

```
class FaceDetector:
```

```
    def __init__(self, model_path, img_size=640, conf_thresh=0.15):
        self.model = YOLO(model_path, task="detect")
        self.img_size = img_size
        self.conf_thresh = conf_thresh
```

```
    def detect(self, frame):
```

```
        # Frame'i alır, sadece conf_thresh üzerindeki kutuları (boxes) döndürür.
```

```
        results = self.model.predict(frame, imgsz=self.img_size, conf=self.conf_thresh, verbose=False)
```

```
        return results[0].boxes
```

face_recognizer.py (Yalnızca Recognition ve Metrik İşlemleri)

```
import numpy as np
```

```
from insightface.app import FaceAnalysis
```

```
class FaceRecognizer:
```

```
    def __init__(self, model_name="buffalo_s", det_size=(160, 160)):
```

```
        self.app = FaceAnalysis(name=model_name, root=".", allowed_modules=["detection", "recognition"])
```

```
        self.app.prepare(ctx_id=-1, det_size=det_size)
```

```
        self.recognition_model = self.app.models["recognition"]
```

```
    def get_embedding(self, aligned_face):
```

```
        # Hizalanmış yüzü alır, embedding vektörünü normalize edip döndürür.
```

```
        emb = self.recognition_model.get_feat(aligned_face)[0]
```

```
return emb / np.linalg.norm(emb)
```

```
def calculate_similarity(self, emb1, emb2, metric="cosine"):
    # İleride Euclidean test etmek isterseniz burayı değiştirmek çok kolay olur.
    if metric == "cosine":
        return np.dot(emb1, emb2)
    elif metric == "euclidean":
        return np.linalg.norm(emb1 - emb2)
```

benchmark.py (Testin Çalıştırıldığı Yer)

```
import cv2
import time
from face_detector import FaceDetector
from face_recognizer import FaceRecognizer
```

Bu dosya kamerayı değil, videoyu okur ve sınıfları çağırarak FPS/Accuracy ölçer.

```
def run_benchmark(video_path, detector, recognizer):
    cap = cv2.VideoCapture(video_path)
    # Çıkarım sürelerini, doğru/yanlış tespitleri tutacak sayaçlar...
    # Döngü içerisinde model testleri...
    pass
```

Bu şekilde bir mimari kurduğunuzda, örneğin girdi boyutunu test etmek istediğinizde sadece FaceRecognizer(det_size=(320, 320)) şeklinde parametreyi değiştirmeniz yeterli olacaktır. Tüm kodun içinde değişken aramanıza gerek kalmaz.

Akıllı Eriřim Kontrol Sistemi: Performans ve Optimizasyon Test Planı

Bu plan, geliřtirilen yüz tanıma sisteminin hem yazılımsal doęruluęunu hem de donanım (Raspberry Pi 5) üzerindeki alıřma verimlilięini ölçmek amacıyla dört ana faza ayrılmıřtır.

Faz 1: Face Detection (Yüz Tespiti) Baseline Metriklerinin ıkarılması

Bu ařamada, modelin hiçbir optimizasyon (Quantization/Pruning) uygulanmamıř orijinal hali (Baseline) test edilecektir.

- **Deęerlendirilecek Model:** Eęitilmiř YOLOv11 (Unquantized / FP32 veya FP16).
- **Ölülecek Metrikler:**
 - **Precision (Kesinlik):** Modelin yüz dedięi řeylerin ne kadarı gerekten yüz?
 - **Recall (Duyarlılık):** Görüntüdeki gerek yüzlerin ne kadarını bulabildi?
 - **F1-Score:** Precision ve Recall deęerlerinin harmonik ortalaması.
 - **mAP@0.5 & mAP@0.5:0.95:** Object Detection modelleri için en temel bařarı metrikleri.
- **Platform:** PC / GPU ortamı (Maksimum kapasitesini görmek için).

Faz 2: Model Optimizasyonu (Pruning & Quantization) ve Karřılařtırma

Bu fazda model sıkıřtırılacak ve hızlandırılacaktır. Her bir optimizasyon adımı sonrası Faz 1'deki metrikler tekrar hesaplanıp Baseline ile kıyaslanacaktır.

- **Adım 2.1 - Pruning (Budama):** Modeldeki aęırlıkların seyreltilmesi (Sparsity) iřlemi. Pruning sonrası mAP düşüřü gözlemlenecek.
- **Adım 2.2 - Quantization (Nicemleme):** Model aęırlıklarının INT8 formatına dönüřtürölmesi.
- **Deęerlendirme:** FP32 (Orijinal) vs. Pruned vs. INT8 modelleri arasında **Accuracy vs. Efficiency** (Doęruluk ve Verimlilik) grafikleri (Trade-off) oluřturulacaktır.

Faz 3: Face Recognition (Yüz Tanıma) Optimizasyonları

Yüz tespiti yapıldıktan sonra alıřan InsightFace ardışık düzeninin (Pipeline) farklı parametrelerle test edilmesi.

- **Embedding Modellerinin Kıyaslanması:** Farklı aęırlıklara sahip modellerin (örneğin InsightFace buffalo_s vs. buffalo_l veya MobileFaceNet vs. ResNet) tanıma bařarısı ve hız (Inference Time) aısından karřılařtırılması.
- **Input Size (Girdi Boyutu) Denemeleri:** YOLO'dan kırpılan (Crop) yüz görüntülerinin ve InsightFace det_size parametresinin (örneğin 160x160 vs. 320x320) performansa etkisinin incelenmesi.
- **Distance/Similarity (Mesafe/Benzerlik) Metrikleri:** Sadece **Cosine Similarity** (Kosinüs Benzerlięi) ile yetinmeyip, alternatif olarak **Euclidean Distance (L2 Norm)** hesaplamalarının yapılması ve hangisinin Threshold (Eřik) belirlemede daha istikrarlı olduęunun kanıtlanması.

Faz 4: Edge Deployment ve Donanım (Raspberry Pi 5) Benchmarkları

Tüm modeller (Orijinal, Pruned, INT8) ve Recognition kombinasyonları Raspberry Pi 5 üzerinde sırayla alıřtırılacak ve gerek dünya donanım metrikleri toplanacaktır.

- **Inference Time (ıkarım Süresi):** Tek bir kare (Frame) için milisaniye (ms) cinsinden geen süre.
- **FPS (Frames Per Second):** Sistemin saniyede iřleyebildięi kare hızı (Detection FPS vs. End-to-End Pipeline FPS).
- **Resource Utilization (Kaynak Tüketimi):** CPU kullanımı (%), RAM (Memory Footprint) tüketimi.
- **Thermal Throttling (Termal Darboęaz):** Sürekli alıřmada RPi5 iřlemci sıcaklıęının FPS üzerindeki negatif etkisi.

Yüz Tanıma (Face Recognition) Test ve Optimizasyon Kontrol Listesi

1. Modellerin Kıyaslanması (Embedding Models)

InsightFace kütüphanesi içerisinde farklı ağırlıklara ve mimarilere sahip modeller bulunur. Bu modellerin özellik çıkarma (Feature Extraction) başarılarını ve hızlarını kıyaslamanız gerekir.

- [] **Model 1: buffalo_s Testi:** Mevcut kodunuzda kullandığınız bu model, hafif yapısıyla uç cihazlar (Edge Devices) için optimize edilmiştir. Temel (Baseline) olarak kullanılacak.
- [] **Model 2: buffalo_l Testi:** Daha derin bir mimariye (ResNet) sahip olan ağır model. Bu modeli, maksimum doğruluğun ne olduğunu görmek ve buffalo_s ile arasındaki hız/başarı farkını (Trade-off) ölçmek için bir referans noktası olarak test edin.
- [] **Model 3: MobileFaceNet Testi:** buffalo_s modelinden bile daha hafif olan ve özellikle mobil/gömülü sistemler için tasarlanmış bu mimariyi, maksimum hız gerektiren senaryolar için test edin.

2. Girdi Boyutlarının (Input Sizes) Kıyaslanması

Yüz tespit (Detection) adımından gelen kırpılmış yüz görüntüsünün (Crop) tanıma modeline girmeden önceki boyutu, hem işleme süresini hem de çıkarılan özelliklerin (Embeddings) kalitesini doğrudan etkiler.

- [] **Boyut 1 (112x112):** Standart InsightFace hizalama (Alignment) boyutudur. Optimum hız sunar.
- [] **Boyut 2 (160x160):** Kırpılmış yüz detaylarını daha fazla koruyarak embedding kalitesini artırıp artırmadığını test edin.
- [] **Boyut 3 (320x320):** Yüksek çözünürlük. Bu test, Raspberry Pi üzerinde belirgin bir darboğaz (Bottleneck) yaratıp yaratmadığını gözlemlemek için yapılmalıdır.

3. Mesafe ve Benzerlik Metrikleri (Distance/Similarity Metrics)

Çıkarılan vektörlerin (Embeddings) veritabanındaki kayıtlı vektörlerle ne kadar uyduğunu matematiksel olarak kanıtlamak için tek bir yöntemle bağlı kalmayın.

- [] **Cosine Similarity Hesaplaması:** İki vektör arasındaki açıyı ölçer. Mevcut kodunuzda uyguladığınız yöntemdir.
- [] **Euclidean Distance (L2 Norm) Hesaplaması:** İki vektör arasındaki doğrudan geometrik mesafeyi ölçer. Formülü şu şekildedir:
$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$
- [] **Eşik Değeri (Threshold) Optimizasyonu:** Her iki metrik için, yetkisiz girişleri sıfıra indirecek en güvenli eşik değerini (örneğin Cosine için > 0.50 , L2 için < 1.0) deneyler yaparak belirleyin ve tabloya dökün.

4. Performans ve Başarı Metriklerinin Hesaplanması

Yüz tanıma modellerini değerlendirirken akademik olarak geçerliliği olan aşağıdaki metrikleri kullanmalısınız. Hazırlayacağınız 2 dakikalık test videosu üzerinden bu değerleri hesaplayın:

- [] **TAR (True Acceptance Rate) Hesabı:** Sistemin kayıtlı olan kişileri başarıyla tanıma ve içeri alma oranı.
- [] **FAR (False Acceptance Rate) Hesabı:** Sistemin kayıtlı olmayan yabancı birini yanlışlıkla tanıyıp içeri alma oranı. (Erişim kontrol sistemlerinde bu değer 0.0 veya buna çok yakın olması hedeflenir).
- [] **EER (Equal Error Rate) Hesabı:** Yanlış kabul (FAR) ve yanlış ret (FRR - False Rejection Rate) oranlarının birbirine eşit olduğu nokta. Bu değer ne kadar düşükse, model o kadar güvenilir.
- [] **Inference Time (Çıkarım Süresi) Ölçümü:** Raspberry Pi üzerinde, 1 adet yüz fotoğrafından vektör (Embedding) çıkarılmasının milisaniye (ms) cinsinden ortalama süresini her model ve girdi boyutu için kaydedin.

Bu checklist'i tamamladığınızda, tezinizde "*Neden buffalo_s modelini ve Cosine Similarity'yi X eşik değeriyle kullandık?*" sorusunu matematiksel ve donanımsal kanıtlarla, sağlam bir temele oturtmuş olacaksınız.